

Week 3 Exercises: Multi-Screen Apps and Navigation

Learning Objectives

After completing this exercise, you should be able to:

- Design a mobile application with multiple screens
 - Explain the difference between screens and reusable components
 - Use React Navigation in a React Native application
 - Navigate between screens using stack navigation
 - Pass data between screens using route parameters
 - Use local JSON data as application content
 - Use React Native Paper components in a multi-screen application
 - Recognize common mobile navigation patterns
 - **The use of AI is allowed in this exercise**
-

Part 1: Mobile Navigation

Answer the following questions briefly:

1. What is the difference between a **screen** and a reusable **component**?
 2. What is stack navigation and how does it work?
 3. What are route parameters and why are they useful?
 4. What is the purpose of back navigation?
 5. Describe three common mobile navigation patterns, such as:
 - Stack navigation
 - Bottom tab navigation
 - Drawer navigation
 6. When would you choose bottom tabs instead of stack navigation?
-

Part 2: Create a Multi-Screen Application

Create a React Native application with at least **three different screens**.

Choose your own application topic or extend your previous homework. The application could present information about products, events, movies, books, recipes, places, hobbies or another topic of your choice.

The application must contain at least:

- a main/home screen
- a list or content screen

- a details screen

Plan the navigation between the screens before implementing the application.

A possible application structure is:

```
App
|
+-- Home
|
+-- Items
    |
    +-- Details
```

Part 3: React Navigation

Install React Navigation and the required dependencies in your Expo project.

Use **Native Stack Navigation** to implement navigation between your screens.

Your application must demonstrate:

- a navigation container
- at least three screens
- navigating from one screen to another
- back navigation

Organize the screens into separate files.

A possible project structure is:

```
project/
|
|-- App.js
|
|-- screens/
|   |-- HomeScreen.js
|   |-- ListScreen.js
|   +-- DetailsScreen.js
|
+-- components/
    +-- ItemCard.js
```

Keep reusable UI elements in `components/` and complete application views in `screens/`.

Part 4: Local Data

Create a small local dataset for your application.

Store the data in a separate JSON file:

```
data/
  data.json
```

The dataset should contain several items and meaningful properties related to your application topic.

Display multiple items from the JSON data on one of your screens.

Use a suitable React Native component such as `FlatList` to display the data.

Do not create separate JSX code manually for every item. Generate the displayed content from the data.

Part 5: Route Parameters and Details Screen

The user must be able to select an item from the list and open its details screen.

Pass information about the selected item to the next screen using **route parameters**.

The Details screen must display information about the selected item.

The navigation flow should work approximately like this:

```
Local JSON data
  |
  v
List Screen
  |
  select item
  |
  | route parameter
  v
Details Screen
```

The Details screen must work with different items. Do not create a separate screen for every item.

Part 6: UI with React Native Paper

Use React Native Paper to create a consistent user interface.

Use suitable Paper components in your application, such as:

- `AppBar`
- `Card`
- `Button`
- `List`

- Icon
- Divider

Use a **Paper AppBar** on at least one screen.

The user interface should make it visually clear:

- which screen the user is currently viewing
- which elements can be selected
- how the user can navigate back

Pay attention to spacing, typography and consistency between screens.

Part 8: Short Reflection

Answer briefly:

1. What is the difference between a screen and a component in your application?
 2. How does your application pass information from one screen to another?
 3. Why is the list generated from data instead of writing each item manually?
 4. What role does React Navigation have in your application?
-

Submission

Submit:

- complete source code
- screenshot(s) showing the different screens
- answers to Parts 1 and 7

Minimum Requirements

Your application must:

- run locally using Expo
- contain at least three screens
- use React Navigation
- use Native Stack Navigation
- support navigation and back navigation
- use route parameters
- load application content from a local JSON file
- display multiple items from data
- contain reusable components
- organize screens and components into separate files
- use React Native Paper
- use a Paper AppBar